ОТЧЁТ О ВЫПОЛНЕНИИ ЗАДАНИЯ НА КУРСОВОЙ ПРОЕКТ

Колледж Astana IT University, Астана, 2026.

Дисциплина: ПМ.07 «Разработка мобильных приложений».

Тема курсового проекта: «Создание приложения для автоматического формирования персонального плана обучения» (приложение «Маршрут»).

Выполнил: студент 3 курса, группа ПО2410, Баринов Николай Владимирович

Руководитель: Синтиков Е. К.

В этом отчёте приложение «Маршрут» сверено с заданием на курсовой проект (КР 2, файл «КР_2_1»). Для каждого пункта задания написано, как он выполнен, где это находится в коде, как показать на защите и какой у него статус. Всё проверено 04.11.2026 запуском программы и тестов, а не по старым документам. Результаты проверки (числа тестов, журнал запросов к серверу) приведены в разделе 5 и в приложении А.

Статусы в таблицах: «выполнено» — проверено запуском; «выполнено, показать на телефоне» — сделано, но проверить можно только на настоящем телефоне; «сделать самому» — шаг, который остаётся студенту перед защитой.

1. Готовность мобильного приложения

Таблица 1 — Готовность приложения

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Приложение полностью реализовано и работоспособно	Есть все экраны, которые были в плане: заставка, регистрация, вход, онбординг из трёх шагов (направление, тест, расписание), предпросмотр плана, главная, план, мои планы, занятие, Pomodoro, темы и свои темы, заметки, статистика, профиль, редактирование профиля, смена пароля, удаление аккаунта. Приложение работает с сервером на ноутбуке	экраны — `lib/presentation/screens/`, переходы — `lib/core/router/app_router.dart`	Пройти сценарий из раздела 7 на телефоне	выполнено, показать на телефоне
Все заявленные функции реализованы и протестированы	Все функции покрыты автоматическими тестами: алгоритм плана, проверка полей, хэширование паролей, клиент API, сервер API, экраны и сквозной сценарий «Заставка → Регистрация → Вход → Главная → План → Профиль → Выход». При проверке 04.11.2026 прошли все 137 тестов из 137	`test/` (5 папок), `integration_test/app_test.dart`	Открыть на ноутбуке терминал и запустить `flutter test` — в конце будет «All tests passed!»	выполнено

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Нет критических ошибок	Анализатор кода не нашёл ни одного замечания, все тесты зелёные, сервер правильно отвечает на 37 проверочных запросов. При этой проверке ошибок, мешающих пользоваться приложением, не найдено. Ошибки, найденные раньше, исправлены и записаны в журнал ошибок	журнал ошибок — `docs/coursework/bugs_log.md`, тест-кейсы — `docs/coursework/testing.md`	Показать, что после выключения сервера приложение не падает, а пишет «Нет связи с сервером»	выполнено

2. Работа с базой данных и API

Таблица 2 — База данных и API

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Развёрнута база данных (локальная или серверная)	База SQLite на собственном сервере. Сервер запускается на ноутбуке и сам создаёт файл базы `marshrut_server.sqlite`. В базе девять таблиц: пользователи, планы, темы плана, занятия, свои темы, заметки, избранное, журнал Pomodoro и токены входа	таблицы — `lib/data/db/app_database.dart`, запуск сервера — `bin/marshrut_server.dart`	Запустить сервер командой `dart run bin/marshrut_server.dart` (или «Запустить сервер» в Windows) и показать строку «База данных: …» в его окне	выполнено
Доступ к данным только через API	Приложение не открывает базу само. Все репозитории в приложении — это классы, которые отправляют HTTP-запросы серверу (`ApiAuthRepository`, `ApiPlanRepository`, `ApiNoteRepository` и другие). Экраны работают с базой только через сервер	клиент — `lib/data/api/api_client.dart`, репозитории — `lib/data/api/`, подключение — `lib/presentation/providers/providers.dart`	Поставить окно сервера рядом с телефоном: после каждого действия в приложении в окне сервера появляется строка запроса	выполнено

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Получение данных	Сервер отдаёт списки и отдельные записи: планы, занятия, темы, заметки, профиль, статистику. Проверено: `GET /api/notes` и `GET /api/notes/1` ответили 200	обработчики — `lib/server/api_server.dart` (`_listNotes`, `_getNote` и др.)	Открыть «Мои заметки» — в окне сервера `GET /api/notes → 200`	выполнено
Добавление данных	Регистрация, новый план, новая заметка, своя тема, запись Pomodoro. Проверено: `POST /api/notes` и `POST /api/custom-topics` ответили 201 с номером новой записи	`_createNote`, `_createCustomTopic`, `_createPlan` в `lib/server/api_server.dart`	Создать заметку — в окне сервера `POST /api/notes → 201`	выполнено
Редактирование данных	Изменение заметки, своей темы, профиля, пароля, названия и настроек плана, отметка занятия. Проверено: `PUT /api/notes/1` и `PUT /api/custom-topics/custom_1` ответили 204, повторное чтение вернуло новый текст	`_updateNote`, `_updateCustomTopic`, `_renamePlan` в `lib/server/api_server.dart`	Изменить заметку — `PUT /api/notes/… → 204`	выполнено

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Удаление данных	Удаление заметки, своей темы, плана со всеми занятиями и аккаунта со всеми данными. Проверено: `DELETE` заметки и своей темы ответили 204, после этого запись не находится (404)	`_deleteNote`, `_deleteCustomTopic`, `_deletePlan`, `_deleteAccount` в `lib/server/api_server.dart`	Удалить заметку — `DELETE /api/notes/… → 204`	выполнено
Корректная обработка ответов API и ошибок	Сервер отвечает понятными кодами: 200, 201, 204 — успех; 400 — неверные данные; 401 — нет входа или токен устарел; 404 — запись не найдена или чужая; 500 — внутренняя ошибка. Приложение показывает текст ошибки внизу экрана, при 401 возвращает на экран входа, если сервер выключен — пишет «Нет связи с сервером», если сервер не ответил за 10 секунд — «Сервер … не отвечает». Проверено запросами (приложение А) и тестами клиента API и экранов	ответы сервера — `_error` и обработка исключений в `lib/server/api_server.dart`; разбор ответов и таймаут — `lib/data/api/api_client.dart`	Остановить сервер (Ctrl+C) и попробовать сохранить заметку — появится красное сообщение. Запустить сервер снова — всё работает	выполнено

3. Пользовательский интерфейс

Прототипы экранов были сделаны до разработки, для защиты концепции: кликабельные макеты 16 экранов в файле `docs/concept/wireframes.html` (открывается в браузере) и схема переходов в `docs/concept/user_flow.md`. Ссылки на проект в Figma в репозитории нет. [ПРОВЕРИТЬ: ссылка на макет Figma — если макет в Figma есть, вставить ссылку сюда; если нет, уточнить у руководителя, засчитываются ли эти макеты как прототип]

Сравнение сделано так: каждый экран макета открыт в браузере и сфотографирован, рядом поставлен снимок того же экрана приложения (снимки приложения делаются автоматически командой `flutter test tool/screenshots/screenshots_test.dart`). Пары картинок лежат в папке `docs/coursework/prototype_compare/`: слева макет, справа приложение.

Таблица 3 — Требования к интерфейсу

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Структура экранов	Все 16 экранов макета есть в приложении, с теми же названиями и тем же содержанием. В приложении есть ещё несколько экранов, которых нет в макете, потому что они понадобились по заданию 2 семестра: заметки, своя тема, настройки плана, смена пароля, удаление аккаунта	`lib/presentation/screens/`	Показать макет в браузере и тот же экран на телефоне	выполнено
Расположение элементов	Основные элементы стоят на тех же местах: заголовок вверху, кольцо прогресса и сегодняшние занятия на главной, поиск и фильтры над списком тем, круглый таймер в центре Pomodoro, кнопка действия внизу. Отличия перечислены в таблице 4	там же	Сравнить пары картинок	выполнено, есть небольшие отличия
Логика навигации	Порядок переходов как в схеме: Заставка → Вход или Главная → Регистрация → Онбординг (направление, тест, расписание) → Предпросмотр → Главная. Пять вкладок внизу с теми же названиями: Главная, План, Темы, Статистика, Профиль. Закрытые экраны без входа не открываются	`lib/core/router/app_router.dart`, нижняя панель — `lib/presentation/screens/shell/main_shell.dart`	Пройти переходы на телефоне по схеме	выполнено

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Цветовая схема и типографика	Цвета взяты из макета без изменений: тёмно-синий `#1A3A8F`, тёмный фон `#0F1D3D`, оранжевый `#F57C00` для кнопок и активной вкладки. Шрифт системный, как в макете (на Android — Roboto); заголовки жирные, как в макете	`lib/core/theme/app_theme.dart` (класс `AppColors`)	Показать главную в светлой и тёмной теме	выполнено
Нет самовольных изменений без обоснования	Все отличия от макета перечислены в таблице 4, у каждого есть причина	—	Ответить по таблице 4, если спросят	выполнено, проверить ссылку на Figma

Таблица 4 — Экраны: макет, снимок приложения и отличия

№	Экран макета	Снимок приложения	Отличия и причина
1	Splash	01_splash	Нет кнопки «Далее (автопереход)»: в макете она только изображала автоматический переход, в приложении переход происходит сам. Вместо надписи «Проверяем сессию…» крутится индикатор загрузки

№	Экран макета	Снимок приложения	Отличия и причина
2	Вход	02_login	Добавлены значки в полях, кнопка «показать пароль» и кнопка «Сервер: …» внизу — она нужна, чтобы ввести адрес сервера на ноутбуке (появилась вместе с сервером API)
3	Регистрация	03_register_validation	Тот же набор полей и кнопок. Добавлен подзаголовок и значки в полях
4	Онбординг: направление	16_direction	Вместо буквы в квадрате — значок направления, добавлено короткое описание направления. Над экраном подписаны шаги «Направление — Тест — Расписание»
5	Онбординг: тест	17_diagnostic	Варианты ответа с кружками выбора, остальное так же
6	Онбординг: расписание	18_schedule	Добавлены «Начало обучения» и переключатель «Есть дедлайн»: срок стал необязательным, потому что не у всех он есть. Количество часов в день подсказывается под ползунком
7	Предпросмотр плана	19_preview	Тот же состав. Предупреждение о дедлайне показывается только когда план не успевает; на снимке план успевает, поэтому предупреждения нет. Кнопка «Изменить расписание» есть ниже, под списком тем
8	Главная	05_home	Карточка прогресса сделана тёмно-синей; добавлен значок Pomodoro в заголовке
9	План	06_plan	Вместо круглой кнопки «Перестроить» внизу — кнопка в баннере «Пропущено N занятий» (как на главной). «Мои планы» открываются значком в заголовке. Добавлена полоса прогресса плана

№	Экран макета	Снимок приложения	Отличия и причина
10	Мои планы	11_plans	Кнопка «Новый план» перенесена в правый нижний угол (плавающая кнопка Material). В карточке добавлены расписание и процент
11	Занятие	14_session	Добавлены блок «Как повторять» для повторений и звёздочка «в избранное». Заметки к теме добавляются кнопкой «Добавить»
12	Pomodoro	12_pomodoro	Добавлен выбор режима 25/5 или 50/10 минут и подсказка о технике Pomodoro
13	Темы	07_topics	Добавлены вкладки направлений сверху; кнопка добавления подписана «Своя тема»
14	Статистика	09_stats	Добавлен график по категориям (круговая диаграмма) и подписи осей
15	Профиль	10_profile	Добавлены карточки «планы, занятия, часы», строка «Сервер API» и выбор темы «Светлая / Авто / Тёмная». Кнопка «Выйти» и «Удалить аккаунт» ниже, видны при прокрутке
16	Редактирование профиля	—	Снимка нет. Поля те же: имя, email, цвет аватара, кнопка «Сохранить». [ПРОВЕРИТЬ: показать экран на телефоне]

При сверке ошибок в интерфейсе, которые нужно исправлять, не найдено: все отличия — дополнения, которые появились по заданиям 2 семестра или для удобства.

4. Установка приложения

Таблица 5 — Установка на смартфон

Подпункт задания	Как выполнено	Где в коде	Как показать на защите	Статус
Установлено на реальный смартфон (Android и/или iOS)	Есть готовые файлы: `marshrut-release.apk` для Android и `marshrut-ios.ipa` для iPhone. Они собираются автоматически на GitHub при каждом изменении (вкладка Actions). Пошаговые инструкции: `docs/Установка_на_iPhone.md` и раздел «Android» в README. При этой проверке установка на телефон не делалась — телефона под рукой не было [ПРОВЕРИТЬ: установить на свой телефон]	сборка — `.github/workflows/build.yml`, иконки — `tool/icons/`	Показать иконку «Маршрут» на экране телефона и открыть приложение	сделать самому
Продемонстрировано в рабочем состоянии (не только эмулятор)	Телефон подключается к серверу на ноутбуке по Wi-Fi: адрес вводится на экране входа кнопкой «Сервер: …». Как подключить, написано в `docs/Сервер_API.md`	экран адреса сервера — `lib/presentation/screens/auth/login_screen.dart`	Пройти сценарий из раздела 7 на телефоне	сделать самому

5. Результаты проверки 04.11.2026

Проверка выполнялась на компьютере с Linux, Flutter 3.47.7 (Dart 3.13.5) — та же версия, на которой собирается проект.

Таблица 6 — Результаты проверки

Что проверялось	Команда	Результат
Получение библиотек	`flutter pub get`	успешно
Оформление кода	`dart format --output=none --set-exit-if-changed .`	96 файлов, изменений 0
Анализатор кода	`flutter analyze`	«No issues found!» — замечаний нет
Все тесты	`flutter test`	137 тестов, прошло 137, не прошло 0, пропущено 0
из них: алгоритм (test/domain)		32 из 32
из них: валидаторы, пароли, правило доступа (test/core)		26 из 26

Что проверялось	Команда	Результат
из них: клиент API и репозитории (test/data)		35 из 35
из них: сервер API (test/server)		23 из 23
из них: экраны и сквозной сценарий (test/presentation)		21 из 21
Снимки экранов	`flutter test tool/screenshots/screenshots_test.dart`	1 тест прошёл, 26 снимков созданы
Интеграционный тест в окне Linux	`flutter test integration_test -d linux`	не запускался: на этом компьютере нет инструментов сборки Linux-приложений (clang++, cmake, ninja, GTK). На GitHub этот тест запускается при каждом изменении
Сервер API вручную	37 запросов curl к `dart run bin/marshrut_server.dart`	все 37 ответов совпали с ожидаемыми (приложение А)

Что показала проверка сервера (полностью — в приложении А):

- заметки: добавление 201, список и одна запись 200, изменение 204, удаление 204, после удаления — 404;

- свои темы: те же четыре операции, те же коды;
- ошибки 400: повторная регистрация с тем же email, неверный пароль при входе, заметка без заголовка, тело запроса не JSON, длительность темы меньше 30 минут;
- ошибки 401: запрос без токена, с неверным токеном, со старым токеном после выхода;
- ошибки 404: удалённая запись, несуществующая запись, несуществующий адрес, чужая заметка (второй пользователь не может ни прочитать, ни удалить заметку первого).

Ошибка 500 вручную не вызывалась: для этого нужно сломать базу. Она проверяется автоматическим тестом сервера («сбой базы»).

Дефектов, которые нужно исправлять в коде, при этой проверке не найдено, поэтому код приложения не менялся.

6. Что сделать самому перед защитой

1. Установить приложение на свой телефон: Android — файл `marshrut-release.apk`, iPhone — `marshrut-ios.ipa` через Sideloadly (инструкция `docs/Установка_на_iPhone.md`). На iPhone бесплатная подпись действует 7 дней — установить не раньше чем за неделю до защиты.
2. Запустить сервер на ноутбуке, подключить телефон к той же сети Wi-Fi и ввести адрес сервера на экране входа (`docs/Сервер_API.md`).
3. Заранее зарегистрироваться и создать план, чтобы на защите были данные.

4. Решить вопрос с Figma: если макет в Figma есть — вставить ссылку в раздел 3; если нет — показать `docs/concept/wireframes.html` и уточнить у руководителя, подходят ли эти макеты.
5. Один раз полностью пройти сценарий защиты из раздела 7 на телефоне, с секундомером.
6. Сделать снимок экрана «Редактировать профиль» на телефоне, если нужен в отчёт.

7. Сценарий защиты (5–7 минут)

Таблица 7 — Сценарий защиты

Время	Что делаю	Что говорю
0:00–0:40	Показываю ноутбук с открытым окном сервера и телефон	«Приложение “Маршрут” составляет личный план обучения. Данные хранятся на моём сервере, он запущен на ноутбуке, база SQLite. Телефон работает с данными только через API по Wi-Fi»
0:40–1:30	Открываю приложение: заставка, вход. Показываю кнопку «Сервер: …»	«Пока видна заставка, приложение загружает каталог с сервера и проверяет вход. Здесь вводится адрес сервера»
1:30–2:30	Вхожу, показываю главную: кольцо прогресса, занятия на сегодня. Отмечаю занятие выполненным	«Отметка ушла на сервер — в окне сервера строка PUT /api/sessions/…/done → 204»

Время	Что делаю	Что говорю
2:30–3:30	Профиль → Мои заметки: создаю заметку, меняю её, удаляю	«Это четыре операции: GET — получение, POST — добавление, PUT — изменение, DELETE — удаление. Справа видны коды ответа 200, 201, 204»
3:30–4:30	Прохожу вкладки План, Темы, Статистика. Рядом открываю `wireframes.html` с тем же экраном	«Экраны сделаны по прототипу: та же структура, те же вкладки, те же цвета — тёмно-синий и оранжевый»
4:30–5:30	Останавливаю сервер (Ctrl+C), пробую сохранить заметку. Запускаю сервер снова и повторяю	«Приложение не падает: пишет “Нет связи с сервером”. После запуска сервера всё работает»
5:30–6:30	Показываю схему архитектуры	«Приложение из трёх слоёв: экраны, логика плана, данные. Слой данных — клиент API. Сервер проверяет токен и данные и только потом обращается к базе»
6:30–7:00	Профиль → Выйти	«При выходе токен удаляется и на сервере. Готов ответить на вопросы»

8. Вероятные вопросы по архитектуре и ответы

Таблица 8 — Вопросы и ответы

Вопрос	Ответ
Как устроено приложение?	Clean Architecture из трёх слоёв. `presentation` — экраны и провайдеры Riverpod. `domain` — сущности и алгоритм плана, он не зависит от Flutter и сервера. `data` — клиент API и репозитории, которые отправляют запросы. Отдельная программа — сервер API на Dart (shelf) с базой SQLite (drift)
Почему приложение не работает с базой напрямую?	По заданию доступ к данным только через API. Так данные живут в одном месте, сервер проверяет права и данные, и одну базу могут использовать несколько устройств
Как приложение узнаёт, кто пользователь?	После входа сервер выдаёт токен. Приложение отправляет его с каждым запросом в заголовке `Authorization: Bearer …`. Токен действует 30 дней, при выходе удаляется на сервере
Как хранятся пароли?	Только в виде хэша PBKDF2-HMAC-SHA256: 100 000 повторений и случайная соль 16 байт. Сам пароль нигде не хранится
Что будет, если сервер выключен?	Клиент API ловит ошибку соединения и показывает «Нет связи с сервером». Если ответа нет 10 секунд — «Сервер не отвечает». Приложение не падает и ничего не теряет
Что значат коды 400, 401, 404?	400 — неверные данные (например, пустой заголовок), 401 — нет входа или токен устарел (приложение отправит на экран входа), 404 — запись не найдена или чужая
Может ли один пользователь увидеть данные другого?	Нет. Сервер берёт пользователя из токена и проверяет, что запись принадлежит ему. Чужая запись отвечает 404 — это проверено запросами и тестом
Как строится план?	Темы образуют граф зависимостей. Известные по тесту темы исключаются, остальные сортируются алгоритмом Кана, раскладываются по учебным дням с лимитом часов, добавляются повторения через 1, 3 и 7 дней и проверяется дедлайн

Вопрос	Ответ
Чем управляется состояние?	Riverpod: экраны подписаны на провайдеры, провайдеры берут данные из репозиториев API. После изменения данных списки обновляются сами
Как проверено, что всё работает?	137 автоматических тестов (все проходят), интеграционный тест настоящего приложения с настоящим сервером, ручная проверка API (37 запросов)
Почему интерфейс отличается от макета в мелочах?	Отличия — дополнения, которые появились во 2 семестре: адрес сервера, заметки, свои темы, выбор темы оформления. Структура, навигация и цвета те же

Приложение А. Проверка работы с данными через API

Проверка выполнена 04.11.2026 на компьютере с Linux. Сервер запущен из папки проекта командой `dart run bin/marshrut_server.dart --port 18093 --db <временный файл>`, то есть с новой пустой базой. Запросы отправлялись программой curl по адресу `http://localhost:18093`. Использовались только тестовые пользователи (test.student@example.com и anna.test@example.com), настоящих данных нет. Токен входа в журнале заменён на `<токен>`.

Итог: 37 запросов, все ответы совпали с ожидаемыми. Заметки и свои темы проверены полностью: добавление (201), получение (200), изменение (204), удаление (204). Ошибки: неверные данные — 400, нет токена, неверный или устаревший токен — 401, запись не найдена или чужая — 404.

Таблица 1 — Сводка запросов

№	Запрос	Что проверялось	Ответ
1	`GET /api/health`	Проверка связи	200
2	`POST /api/auth/register`	Регистрация тестового пользователя	201
3	`POST /api/auth/register`	Повторная регистрация с тем же email — ошибка 400	400
4	`POST /api/auth/login`	Вход с неверным паролем — ошибка	400
5	`POST /api/auth/login`	Вход: сервер выдаёт токен	200
6	`GET /api/notes`	Список заметок без токена — ошибка 401	401
7	`GET /api/notes`	Список заметок с неверным токеном — ошибка 401	401
8	`GET /api/me`	Профиль (получение данных с токеном)	200
9	`POST /api/notes`	Добавление заметки	201
10	`GET /api/notes`	Получение списка заметок	200
11	`GET /api/notes/1`	Получение одной заметки	200
12	`PUT /api/notes/1`	Изменение заметки	204
13	`GET /api/notes/1`	Заметка после изменения	200
14	`POST /api/notes`	Добавление заметки без заголовка — ошибка 400	400
15	`POST /api/notes`	Тело запроса не JSON — ошибка 400	400
16	`DELETE /api/notes/1`	Удаление заметки	204
17	`GET /api/notes/1`	Получение удалённой заметки — ошибка 404	404
18	`PUT /api/notes/1`	Изменение удалённой заметки — ошибка 404	404
19	`DELETE /api/notes/1`	Повторное удаление — ошибка 404	404

№	Запрос	Что проверялось	Ответ
20	`GET /api/notes/999999`	Несуществующая заметка — ошибка 404	404
21	`GET /api/nesushchestvuet`	Несуществующий адрес — ошибка 404	404
22	`POST /api/custom-topics`	Своя тема с длительностью 20 минут (меньше 30) — ошибка 400	400
23	`POST /api/custom-topics`	Добавление своей темы	201
24	`GET /api/custom-topics`	Получение списка своих тем	200
25	`PUT /api/custom-topics/custom_1`	Изменение своей темы	204
26	`GET /api/custom-topics/custom_1`	Своя тема после изменения	200
27	`DELETE /api/custom-topics/custom_1`	Удаление своей темы	204
28	`GET /api/custom-topics/custom_1`	Получение удалённой темы — ошибка 404	404
29	`POST /api/auth/logout`	Выход: токен удаляется на сервере	204
30	`GET /api/me`	Запрос со старым токеном после выхода — ошибка 401	401
31	`POST /api/auth/register`	Регистрация второго тестового пользователя (A)	201
32	`POST /api/auth/login`	Вход пользователя A	200
33	`POST /api/notes`	Пользователь A добавляет заметку	201

№	Запрос	Что проверялось	Ответ
34	`POST /api/auth/login`	Вход первого тестового пользователя (Б)	200
35	`GET /api/notes/2`	Пользователь Б запрашивает заметку пользователя А — ошибка 404	404
36	`DELETE /api/notes/2`	Пользователь Б пытается удалить заметку пользователя А — ошибка 404	404
37	`GET /api/notes`	Список заметок пользователя Б — чужой заметки в нём нет	200

Запросы и ответы полностью

1. Проверка связи

```
GET /api/health

Ответ 200:
{"status":"ok","service":"Маршрут","version":1}
```

2. Регистрация тестового пользователя

```
POST /api/auth/register
Content-Type: application/json

{"name":"Тест
Студент","email":"test.student@example.com","password":"Parol2026"}
```

```
Ответ 201:
{"id":1,"name":"Тест
Студент","email":"test.student@example.com","avatarColor":4282211070,"createdAt
11-
04T05:44:05.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}
```

3. Повторная регистрация с тем же email — ошибка 400

```
POST /api/auth/register
Content-Type: application/json

{"name":"Тест
Студент","email":"test.student@example.com","password":"Parol2026"}

Ответ 400:
{"error":"Пользователь с таким email уже существует"}
```

4. Вход с неверным паролем — ошибка

```
POST /api/auth/login
Content-Type: application/json

{"email":"test.student@example.com","password":"Neverno123"}

Ответ 400:
{"error":"Неверный email или пароль"}
```

5. Вход: сервер выдаёт токен

```
POST /api/auth/login
Content-Type: application/json

{"email":"test.student@example.com","password":"Parol2026"}

Ответ 200:
{"token":"<токен>","user":{"id":1,"name":"Тест
Студент","email":"test.student@example.com","avatarColor":4282211070,"createdAt
11-
04T05:44:05.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}}
```

6. Список заметок без токена — ошибка 401

```
GET /api/notes

Ответ 401:
{"error":"Требуется вход в аккаунт"}
```

7. Список заметок с неверным токеном — ошибка 401

```
GET /api/notes
Authorization: Bearer неверный-токен-123
```

```
Ответ 401:
{"error":"Сессия истекла, войдите снова"}
```

8. Профиль (получение данных с токеном)

```
GET /api/me
Authorization: Bearer <токен>

Ответ 200:
{"id":1,"name":"Тест
Студент","email":"test.student@example.com","avatarColor":4282211070,"createdAt
11-
04T05:44:05.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}
```

9. Добавление заметки

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

{"title":"Списки в Python","body":"append добавляет в
конец","topicId":"py_lists"}

Ответ 201:
{"id":1}
```

10. Получение списка заметок

```
GET /api/notes
Authorization: Bearer <токен>

Ответ 200:
[{"id":1,"userId":1,"title":"Списки в Python","body":"append добавляет в
конец","topicId":"py_lists","topicTitle":null,"createdAt":"2026-11-
04T05:44:05.000","updatedAt":"2026-11-04T05:44:05.000"}]
```

11. Получение одной заметки

```
GET /api/notes/1
Authorization: Bearer <токен>

Ответ 200:
{"id":1,"userId":1,"title":"Списки в Python","body":"append добавляет в
конец","topicId":"py_lists","topicTitle":null,"createdAt":"2026-11-
04T05:44:05.000","updatedAt":"2026-11-04T05:44:05.000"}
```

12. Изменение заметки

```
PUT /api/notes/1
Authorization: Bearer <токен>
Content-Type: application/json

{"title":"Списки в Python","body":"append добавляет в конец, insert — по
индексу"}
```

```
Ответ 204:
(пустое тело)
```

13. Заметка после изменения

```
GET /api/notes/1
Authorization: Bearer <токен>

Ответ 200:
{"id":1,"userId":1,"title":"Списки в Python","body":"append добавляет в конец,
insert — по индексу","topicId":"py_lists","topicTitle":null,"createdAt":"2026-
11-04T05:44:05.000","updatedAt":"2026-11-04T05:44:05.000"}
```

14. Добавление заметки без заголовка — ошибка 400

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

{"body":"без заголовка"}

Ответ 400:
{"error":"Поле «title» не заполнено или имеет неверный тип"}
```

15. Тело запроса не JSON — ошибка 400

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

это не json

Ответ 400:
{"error":"Тело запроса должно быть объектом JSON"}
```

16. Удаление заметки

```
DELETE /api/notes/1
Authorization: Bearer <токен>

Ответ 204:
(пустое тело)
```

17. Получение удалённой заметки — ошибка 404

```
GET /api/notes/1
Authorization: Bearer <токен>

Ответ 404:
{"error":"Заметка не найдена"}
```

18. Изменение удалённой заметки — ошибка 404

```
PUT /api/notes/1
Authorization: Bearer <токен>
Content-Type: application/json

{"title":"x","body":"y"}

Ответ 404:
{"error":"Заметка не найдена"}
```

19. Повторное удаление — ошибка 404

```
DELETE /api/notes/1
Authorization: Bearer <токен>

Ответ 404:
{"error":"Заметка не найдена"}
```

20. Несуществующая заметка — ошибка 404

```
GET /api/notes/999999
Authorization: Bearer <токен>

Ответ 404:
{"error":"Заметка не найдена"}
```

21. Несуществующий адрес — ошибка 404

```
GET /api/nesushchestvuet
Authorization: Bearer <токен>

Ответ 404:
{"error":"Адрес не найден"}
```

22. Своя тема с длительностью 20 минут (меньше 30) — ошибка 400

```
POST /api/custom-topics
Authorization: Bearer <токен>
Content-Type: application/json

{"directionId":"python","title":"Своя
тема","category":"Практика","minutes":20}

Ответ 400:
{"error":"От 0,5 до 100 часов"}
```

23. Добавление своей темы

```
POST /api/custom-topics
Authorization: Bearer <токен>
Content-Type: application/json

{"directionId":"python","title":"Работа с файлами","description":"open, with,
чтение и запись","category":"Практика","minutes":90}
```

```
Ответ 201:
{"id":"custom_1"}
```

24. Получение списка своих тем

```
GET /api/custom-topics
Authorization: Bearer <токен>

Ответ 200:
[{"id":"custom_1","title":"Работа с файлами","description":"open, with, чтение
и запись","minutes":90,"level":3,"category":"Практика","prerequisites":
[],"isCustom":true}]
```

25. Изменение своей темы

```
PUT /api/custom-topics/custom_1
Authorization: Bearer <токен>
Content-Type: application/json

{"title":"Работа с файлами и папками","description":"open, with,
os.listdir","category":"Практика","minutes":120}

Ответ 204:
(пустое тело)
```

26. Своя тема после изменения

```
GET /api/custom-topics/custom_1
Authorization: Bearer <токен>

Ответ 200:
{"id":"custom_1","title":"Работа с файлами и папками","description":"open,
with,
os.listdir","minutes":120,"level":3,"category":"Практика","prerequisites":
[],"isCustom":true,"directionId":"python"}
```

27. Удаление своей темы

```
DELETE /api/custom-topics/custom_1
Authorization: Bearer <токен>

Ответ 204:
(пустое тело)
```

28. Получение удалённой темы — ошибка 404

```
GET /api/custom-topics/custom_1
Authorization: Bearer <токен>

Ответ 404:
{"error":"Тема не найдена"}
```

29. Выход: токен удаляется на сервере

```
POST /api/auth/logout
Authorization: Bearer <токен>

Ответ 204:
(пустое тело)
```

30. Запрос со старым токеном после выхода — ошибка 401

```
GET /api/me
Authorization: Bearer <токен>

Ответ 401:
{"error":"Сессия истекла, войдите снова"}
```

31. Регистрация второго тестового пользователя (А)

```
POST /api/auth/register
Content-Type: application/json

{"name":"Пользователь
Анна","email":"anna.test@example.com","password":"Anna2026x"}

Ответ 201:
{"id":2,"name":"Пользователь
Анна","email":"anna.test@example.com","avatarColor":4281236786,"createdAt":"202
```

```
11-
04T05:44:17.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}
```

32. Вход пользователя A

```
POST /api/auth/login
Content-Type: application/json

{"email":"anna.test@example.com","password":"Anna2026x"}

Ответ 200:
{"token":"<токен>","user":{"id":2,"name":"Пользователь
Анна","email":"anna.test@example.com","avatarColor":4281236786,"createdAt":"202
11-
04T05:44:17.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}}
```

33. Пользователь A добавляет заметку

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

{"title":"Заметка Анны","body":"личная"}

Ответ 201:
{"id":2}
```

34. Вход первого тестового пользователя (Б)

```
POST /api/auth/login
Content-Type: application/json

{"email":"test.student@example.com","password":"Parol2026"}

Ответ 200:
{"token":"<токен>","user":{"id":1,"name":"Тест
Студент","email":"test.student@example.com","avatarColor":4282211070,"createdAt
11-
04T05:44:05.000","reminderEnabled":false,"reminderHour":19,"reminderMinute":0}}
```

35. Пользователь Б запрашивает заметку пользователя А — ошибка 404

```
GET /api/notes/2
Authorization: Bearer <токен>

Ответ 404:
{"error":"Заметка не найдена"}
```

36. Пользователь Б пытается удалить заметку пользователя А — ошибка 404

```
DELETE /api/notes/2
Authorization: Bearer <токен>
```

```
Ответ 404:
{"error":"Заметка не найдена"}
```

37. Список заметок пользователя Б — чужой заметки в нём нет

```
GET /api/notes
Authorization: Bearer <токен>

Ответ 200:
[]
```

Журнал в окне сервера

Так выглядит журнал, который сервер печатает в своём окне: время, метод, адрес, код ответа и время обработки. Первые две строки — проверка связи перед началом.

```
Журнал запросов:
05:44:04  GET    /api/health  →  200  (4 мс)
05:44:04  GET    /api/health  →  200  (0 мс)
05:44:05  POST   /api/auth/register  →  201  (300 мс)
05:44:05  POST   /api/auth/register  →  400  (1 мс)
05:44:05  POST   /api/auth/login  →  400  (215 мс)
05:44:05  POST   /api/auth/login  →  200  (211 мс)
05:44:05  GET    /api/notes  →  401  (0 мс)
05:44:05  GET    /api/notes  →  401  (1 мс)
05:44:05  GET    /api/me  →  200  (1 мс)
05:44:05  POST   /api/notes  →  201  (2 мс)
05:44:05  GET    /api/notes  →  200  (5 мс)
```

```
05:44:05  GET    /api/notes/1  →  200  (1 мс)
05:44:05  PUT    /api/notes/1  →  204  (2 мс)
05:44:05  GET    /api/notes/1  →  200  (0 мс)
05:44:05  POST   /api/notes  →  400  (0 мс)
05:44:05  POST   /api/notes  →  400  (1 мс)
05:44:05  DELETE /api/notes/1  →  204  (1 мс)
05:44:05  GET    /api/notes/1  →  404  (0 мс)
05:44:05  PUT    /api/notes/1  →  404  (0 мс)
05:44:05  DELETE /api/notes/1  →  404  (0 мс)
05:44:05  GET    /api/notes/999999  →  404  (0 мс)
05:44:05  GET    /api/nesushchestvuet  →  404  (0 мс)
05:44:05  POST   /api/custom-topics  →  400  (1 мс)
05:44:05  POST   /api/custom-topics  →  201  (2 мс)
05:44:05  GET    /api/custom-topics  →  200  (2 мс)
05:44:05  PUT    /api/custom-topics/custom_1  →  204  (2 мс)
05:44:05  GET    /api/custom-topics/custom_1  →  200  (1 мс)
05:44:05  DELETE /api/custom-topics/custom_1  →  204  (3 мс)
05:44:06  GET    /api/custom-topics/custom_1  →  404  (0 мс)
05:44:06  POST   /api/auth/logout  →  204  (0 мс)
05:44:06  GET    /api/me  →  401  (0 мс)
05:44:17  POST   /api/auth/register  →  201  (285 мс)
05:44:17  POST   /api/auth/login  →  200  (246 мс)
05:44:17  POST   /api/notes  →  201  (1 мс)
05:44:17  POST   /api/auth/login  →  200  (221 мс)
05:44:17  GET    /api/notes/2  →  404  (1 мс)
05:44:17  DELETE /api/notes/2  →  404  (1 мс)
05:44:17  GET    /api/notes  →  200  (0 мс)
```

Приложение Б. Экраны: прототип и приложение

На каждой картинке слева экран прототипа (docs/concept/wireframes.html), справа тот же экран приложения.

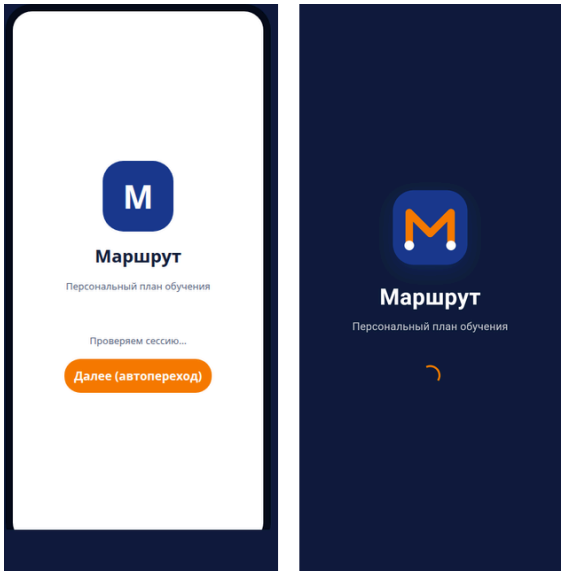


Рисунок Б.1 — Splash

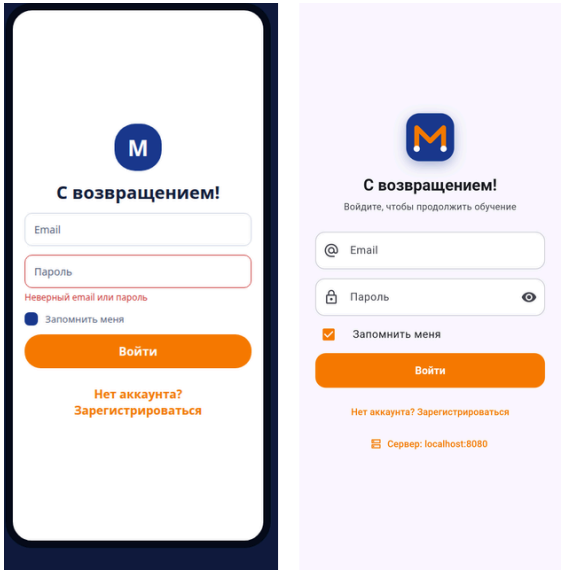


Рисунок Б.2 — Вход

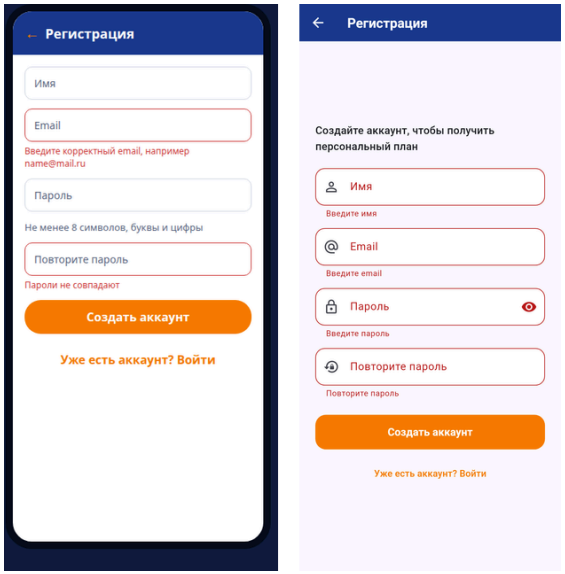


Рисунок Б.3 — Регистрация

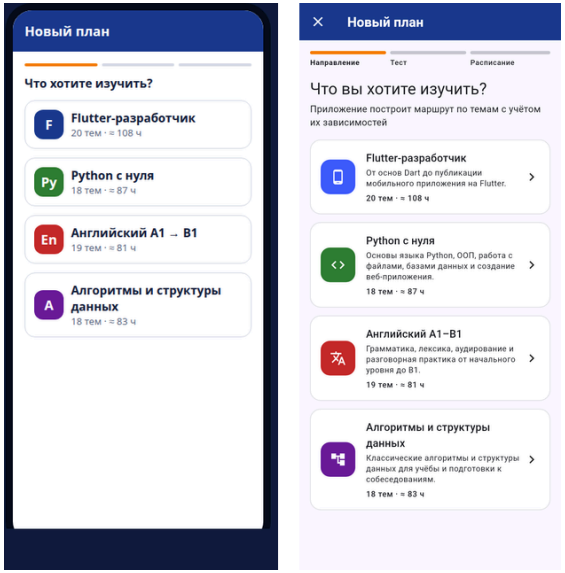


Рисунок Б.4 — Онбординг: направление

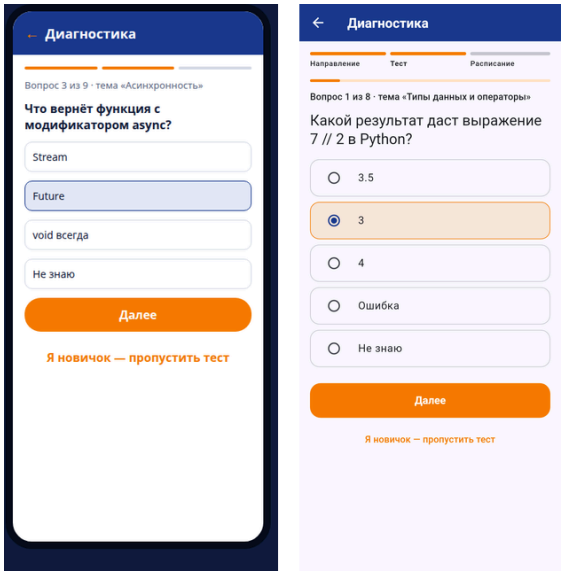


Рисунок Б.5 — Онбординг: тест

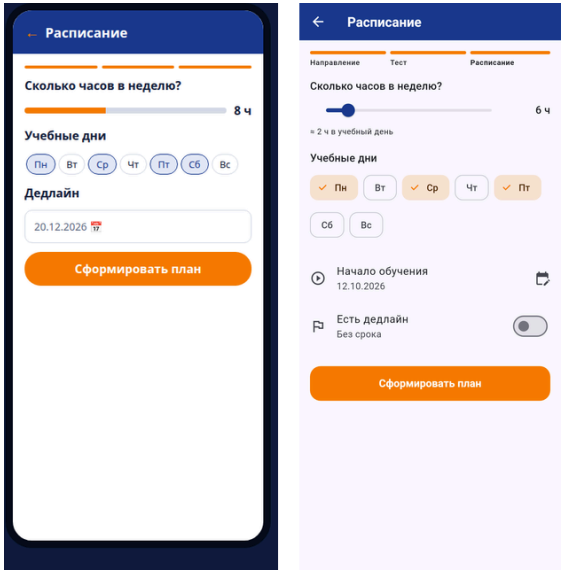


Рисунок Б.6 — Онбординг: расписание

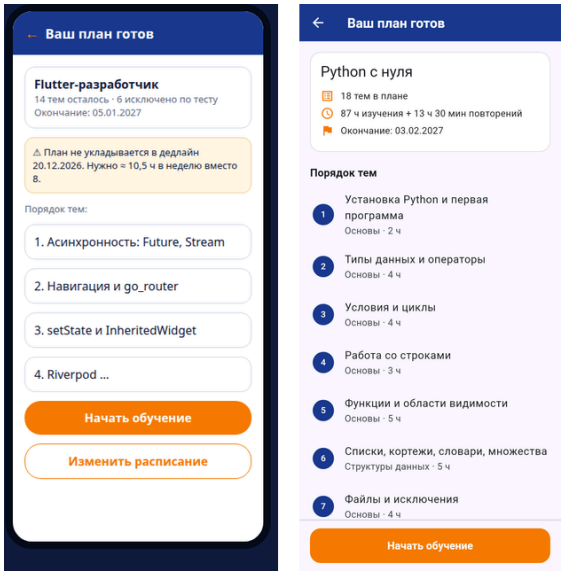


Рисунок Б.7 — Предпросмотр плана

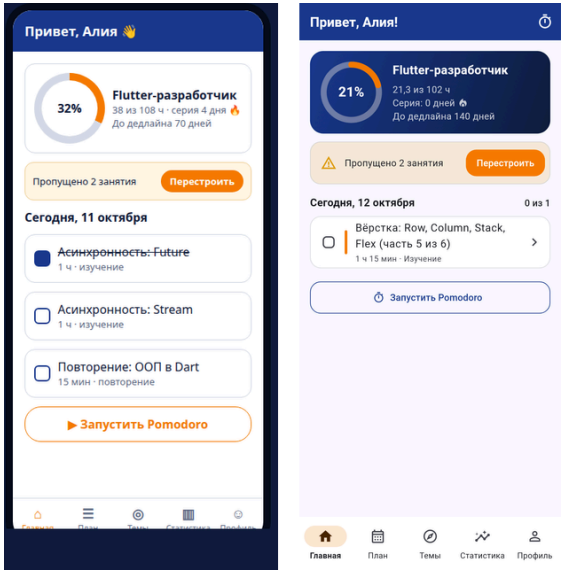


Рисунок Б.8 — Главная

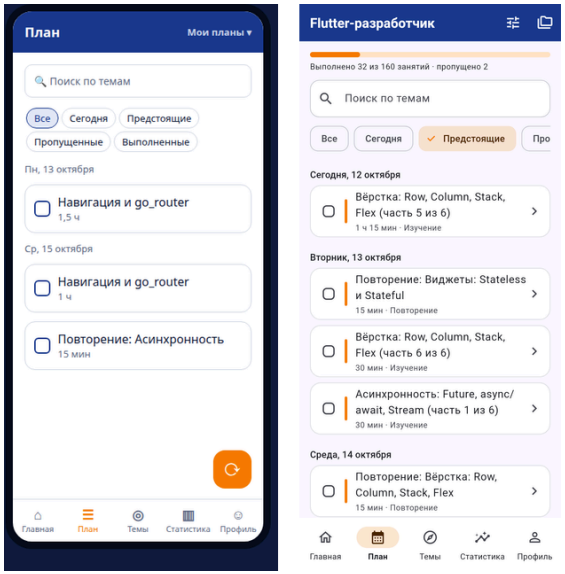


Рисунок Б.9 — План

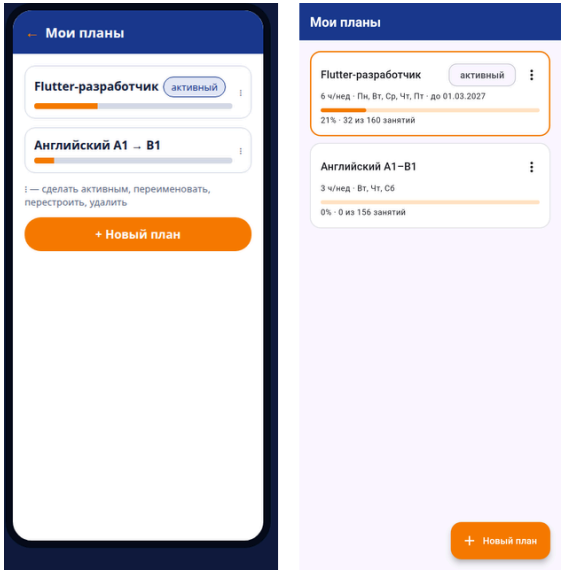


Рисунок Б.10 — Мои планы

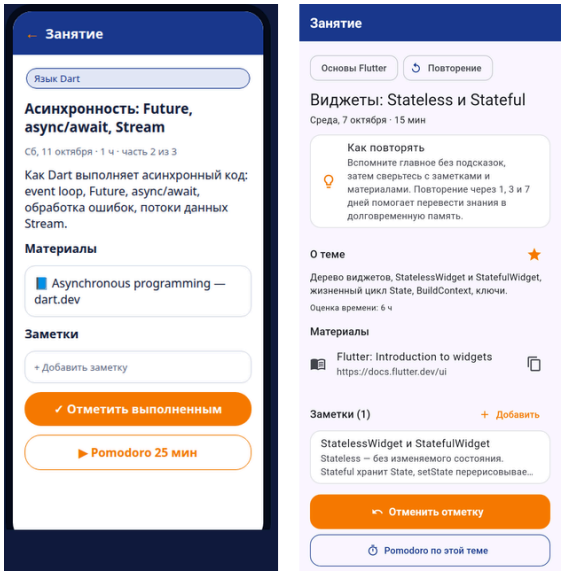


Рисунок Б.11 — Занятие

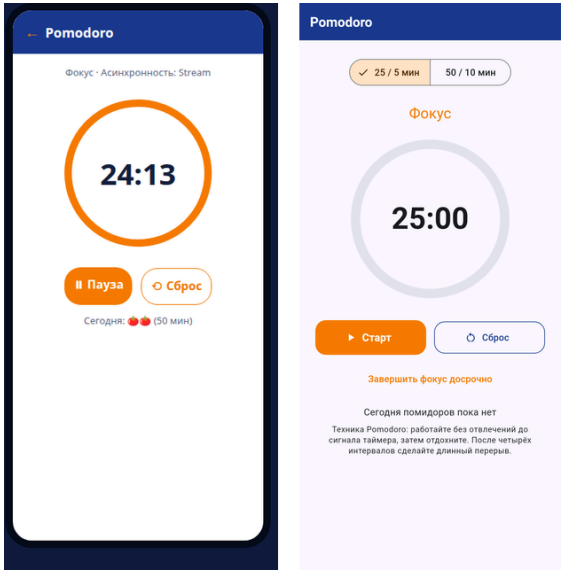


Рисунок Б.12 — Pomodoro

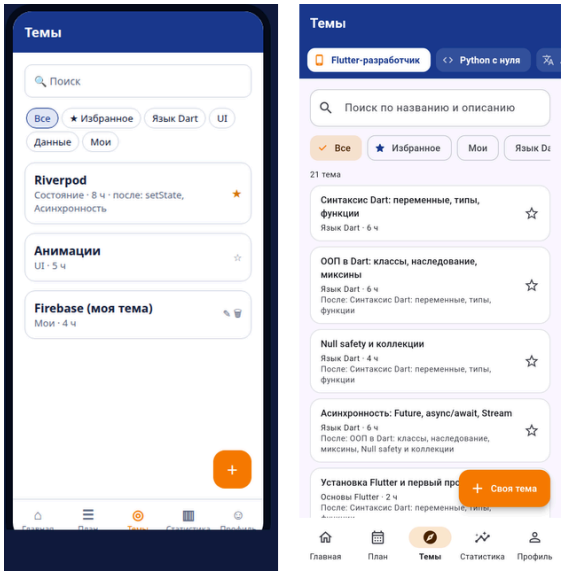


Рисунок Б.13 — Темы

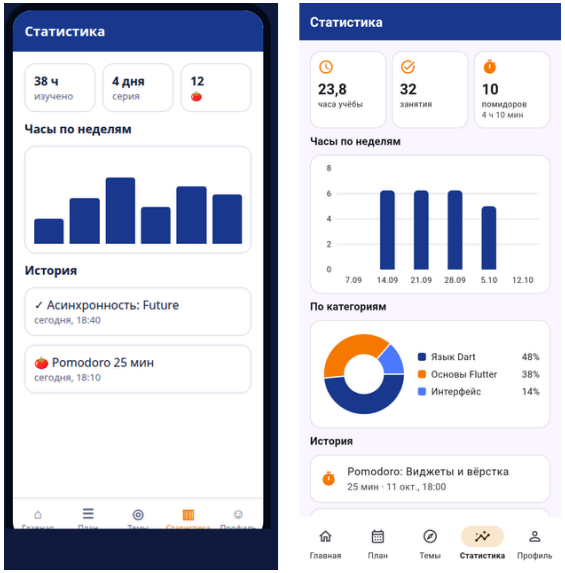


Рисунок Б.14 — Статистика

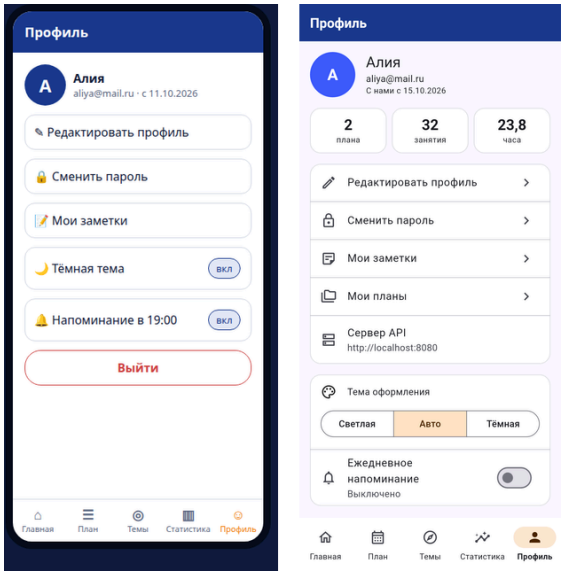


Рисунок Б.15 — Профиль



Рисунок Б.16 — Редактирование профиля (только макет)